ASSIGNMENT 3 - TASK 2

Comparison of FCNN Architectures and Optimization Algorithms on a Five-Class Fashion-MNIST Dataset

Student Name	Sushmita Halder, Jyoti, Rajeev Kumar
Roll Number	2621MA08, 2621MA12, 2621MA01
Group	08
Course	Large Language Models
Submission Date	15/09/2026

1. Introduction

The objective of Task 2 was to study the performance of fully connected neural network (FCNN) architectures under different optimization algorithms for a five-class Fashion-MNIST classification problem. Three FCNN architectures with three, four, and five hidden layers were compared using six optimization methods: SGD, Batch Gradient Descent, SGD with Momentum, Nesterov Accelerated Gradient (NAG), RMSProp, and Adam.

The updated experiment uses the Fashion-MNIST CSV datasets supplied in the notebook. The workflow includes data inspection, selection of five classes, label remapping, pixel normalization, a stratified train-validation split, conversion of arrays to PyTorch tensors, creation of TensorDataset objects, model training, convergence analysis, accuracy comparison, and final evaluation on the held-out test set. The best model is selected using validation accuracy.

2. Dataset and Data Processing

2.1 Dataset

The notebook loads fashion-mnist_train.csv.zip and fashion-mnist_test.csv.zip. The original training file contains 60,000 samples and 785 columns, while the original test file contains 10,000 samples and 785 columns. Each row contains one label column and 784 pixel features corresponding to a 28 × 28 grayscale image.

Dataset	Original Samples	Columns	Selected Classes	Selected Samples
Training	60,000	785	0–4	30,000
Testing	10,000	785	0–4	5,000

2.2 Selected Classes

Five classes were selected: T-shirt/top, Trouser, Pullover, Dress, and Coat. The notebook filters both the training and test data to retain only these classes. This produces 30,000 training samples and 5,000 test samples, with 784 input features after removing the label column.

Original Label	Class Name	Remapped Label
0	T-shirt/top	0
1	Trouser	1
2	Pullover	2
3	Dress	3
4	Coat	4

Label remapping was explicitly included in the preprocessing pipeline. Because the selected labels were already consecutive (0, 1, 2, 3, 4), the resulting mapping is the identity mapping {0:0, 1:1, 2:2, 3:3, 4:4}. This ensures that the labels match the five output neurons of the classifier.

2.3 Data Processing

After filtering the classes, the label column is separated from the 784 pixel features. The resulting feature arrays have shapes (30,000, 784) for training and (5,000, 784) for testing. The labels have shapes (30,000,) and (5,000,), respectively.

The pixel values are stored as integers from 0 to 255. They are converted to float32 and divided by 255.0, giving a normalized range of [0, 1]. Normalization puts the input values on a small and consistent scale, which is suitable for neural-network training. The notebook verifies a minimum of 0.0 and maximum of 1.0 after normalization.

2.4 Stratified Train–Validation Split

The 30,000 selected training samples are divided into 80% training data and 20% validation data using a stratified procedure. The split is performed class-by-class: samples belonging to each class are shuffled, 20% are assigned to validation, and the remaining 80% are assigned to training. This preserves the class distribution in both subsets.

Subset	Samples	Features	Role
Training	24,000	784	Model fitting
Validation	6,000	784	Model selection
Test	5,000	784	Final evaluation

It is confirms an equal class distribution: the training set contains 4,800 samples per class, the validation set contains 1,200 per class, and the test set contains 1,000 per class. Thus, all three subsets remain balanced across the five selected classes.

2.5 Conversion of Data to Tensors and Dataset Creation

The NumPy arrays are converted into PyTorch tensors before model training. Feature tensors use torch.float32, while class labels use torch.long, which is the appropriate integer type for Cross-Entropy Loss. The notebook creates X_train_tensor, y_train_tensor, X_val_tensor, y_val_tensor, X_test_tensor, and y_test_tensor.

Three TensorDataset objects are then created: train_dataset, val_dataset, and test_dataset. Each Tensor Dataset stores the feature tensor together with its corresponding label tensor, making the data convenient to use with PyTorch DataLoader objects during training and evaluation.

Cross-entropy is a loss function that measures the difference between the predicted probability distribution and the actual class. For classification, it penalizes the model strongly when it assigns a low probability to the correct class, and the resulting loss is used during backpropagation to update the neural-network weights

3. FCNN Architectures

All models take 784 input features and produce five output scores, one for each selected clothing class. ReLU activation is used after every hidden linear layer. The three architectures differ only in the number and size of their hidden layers.

Architecture	Hidden Layers	Network Structure
Architecture_1	[128, 64, 32]	784 → 128 → 64 → 32 → 5
Architecture_2	[256, 128, 64, 32]	784 → 256 → 128 → 64 → 32 → 5
Architecture_3	[256, 128, 64, 32, 16]	784 → 256 → 128 → 64 → 32 → 16 → 5

4. Training Setup

Parameter	Value
Loss function	Cross-Entropy Loss
Learning rate	0.001
Convergence threshold	1×10^{-4}
Momentum	0.9
RMSProp alpha	0.99
RMSProp epsilon	1×10^{-8}
Adam β1	0.9
Adam β2	0.999
Adam epsilon	1×10^{-8}
Random seed	42

The training loop performs forward propagation, computes the cross-entropy loss, performs backpropagation using loss backward, and updates model parameters using optimizer step. Average training errors are recorded after each epoch. Training stops when the absolute change in average training error between two successive epochs becomes smaller than 1×10^{-4}.

5. Optimization Algorithms

Six optimization approaches were evaluated: SGD, Batch GD, Momentum, NAG, RMSProp, and Adam. For Batch GD, the batch size is set equal to the complete training dataset, while the other methods use a batch size of 100 in the training loop.

Optimizer	Description in the experiment
SGD	Standard stochastic gradient descent.
Batch GD	SGD update using the complete training set in one batch.
Momentum	SGD with momentum = 0.9.
NAG	SGD with momentum and Nesterov look-ahead.
RMSProp	Adaptive learning rate using alpha = 0.99.
Adam	Adaptive optimizer using $\beta1 = 0.9$ and $\beta2 = 0.999$.

6. Results

6.1 Epochs to Convergence and Accuracy

The model trained all 18 architecture–optimizer combinations. The results below are taken directly from the final results table generated. Accuracy values are reported as percentages and execution time is shown in seconds.

Architecture	Optimizer	Epochs	Train Acc. (%)	Val. Acc. (%)	Time (s)
Architecture_1	SGD	205	88.81	88.12	120.20
Architecture_1	Batch GD	2	20.99	20.78	0.70
Architecture_1	Momentum	59	90.85	89.37	35.52
Architecture_1	NAG	161	96.64	90.98	99.22
Architecture_1	RMSProp	49	97.20	91.52	31.94
Architecture_1	Adam	161	99.74	91.45	107.16
Architecture_2	SGD	203	87.88	87.07	127.62
Architecture_2	Batch GD	2	19.65	19.55	0.50
Architecture_2	Momentum	74	92.58	89.45	48.10
Architecture_2	NAG	68	93.88	90.87	44.10
Architecture_2	RMSProp	31	95.32	91.53	21.90
Architecture_2	Adam	244	100.00	92.05	179.46
Architecture_3	SGD	262	89.32	88.45	179.33
Architecture_3	Batch GD	2	20.00	20.00	0.94
Architecture_3	Momentum	69	93.91	90.47	47.51
Architecture_3	NAG	85	96.29	90.87	59.47
Architecture_3	RMSProp	42	96.47	91.28	31.80
Architecture_3	Adam	25	94.44	90.77	19.09

6.2 Training Error versus Epochs

Average training-error curves were generated separately for the three architectures, with the six optimizer curves shown together. These plots illustrate how quickly the training error changes and how the convergence criterion behaves for different optimizers.

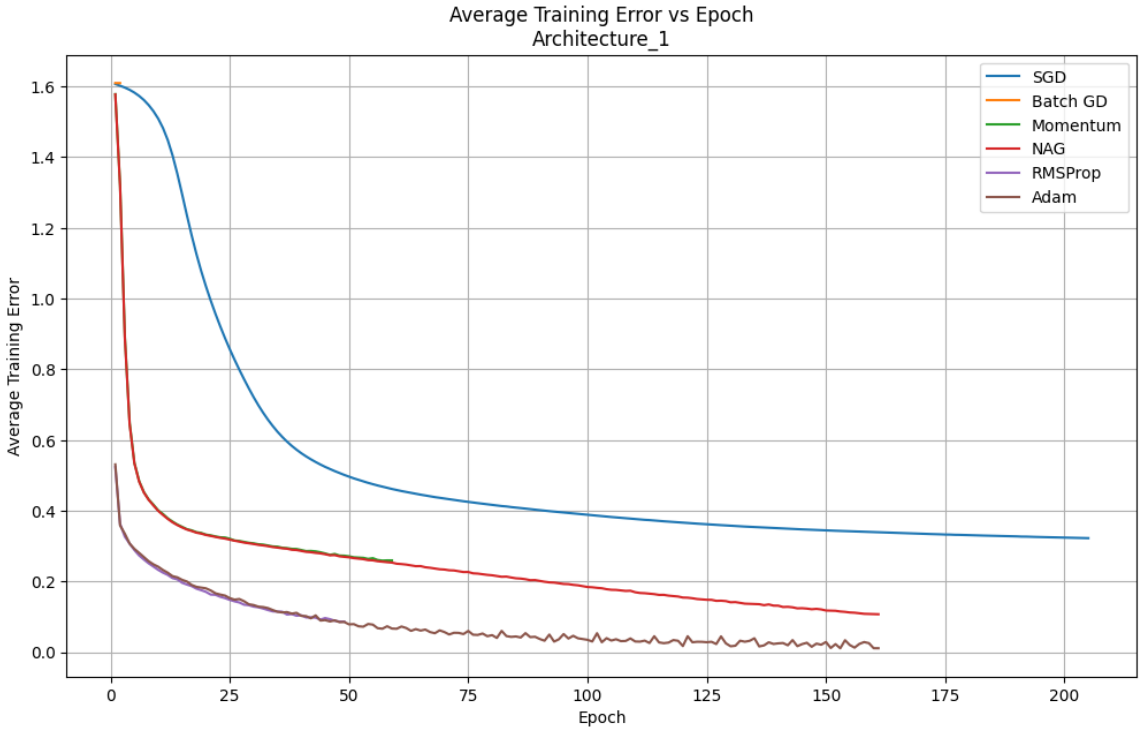


Figure 1: Average training error versus epoch for Architecture_1.

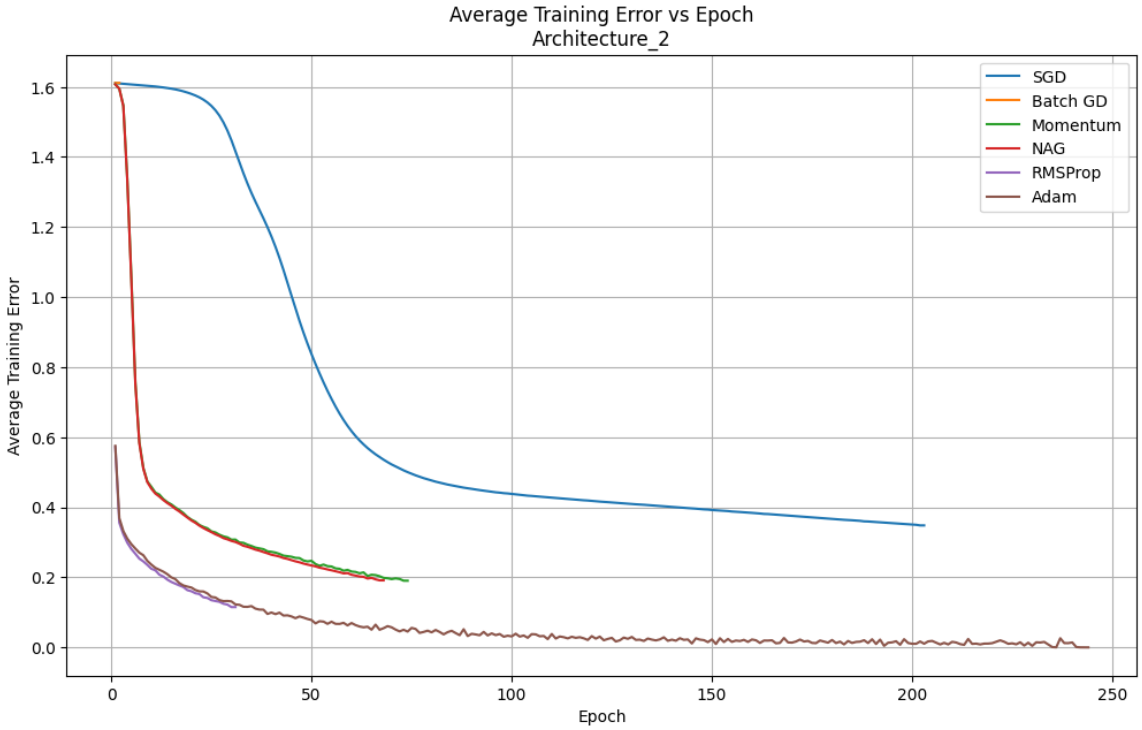


Figure 2: Average training error versus epoch for Architecture_2.

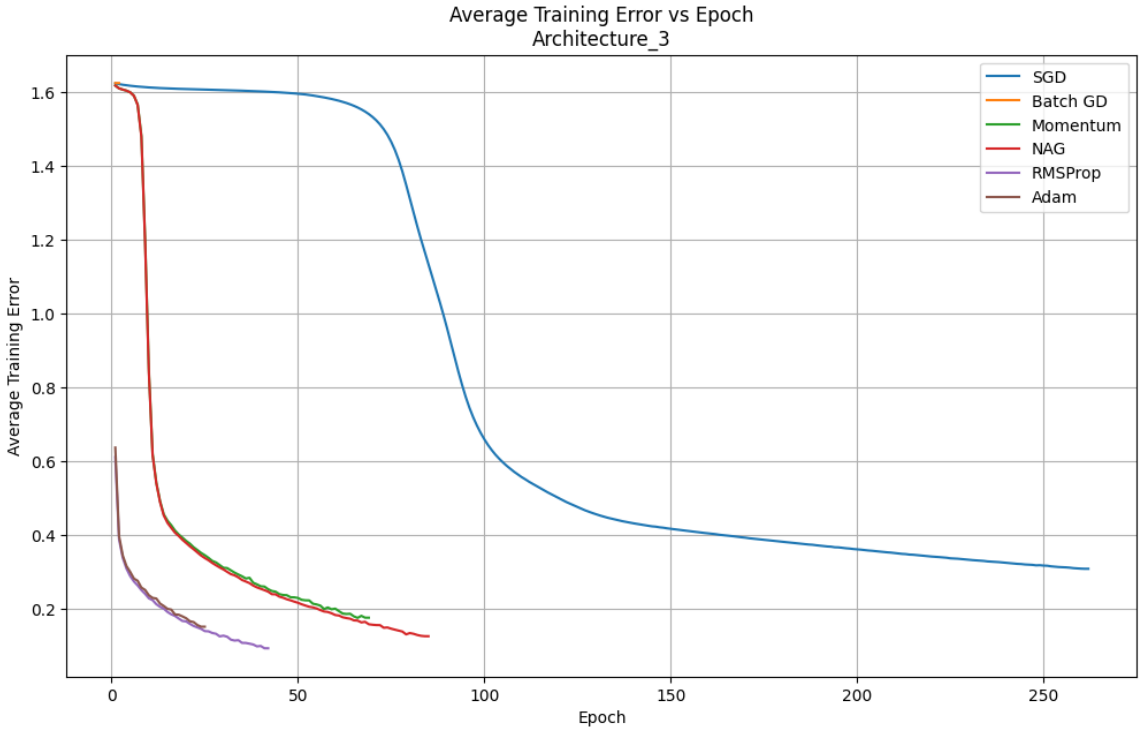


Figure 3: Average training error versus epoch for Architecture_3.

6.3 Selection of the Best Model

Model selection was based on validation accuracy. Architecture_2 with Adam achieved the highest validation accuracy of 92.05% among all 18 configurations. Its training accuracy was 100.00%, and it converged in 244 epochs.

Best Architecture	Hidden Layers	Optimizer	Epochs	Training Accuracy	Validation Accuracy
Architecture_2	[256, 128, 64, 32]	Adam	244	100.00%	92.05%

7. Final Test Evaluation

7.1 Test Accuracy

The selected Architecture_2 model trained with Adam achieved a test accuracy of 92.36% on the 5,000-sample held-out test set. The final summary reports 100.00% training accuracy, 92.05% validation accuracy, 92.36% test accuracy, and 244 epochs to convergence.

7.2 Test Confusion Matrix

Actual \ Predicted	T-shirt/top	Trouser	Pullover	Dress	Coat
T-shirt/top	946	2	25	24	3
Trouser	4	982	4	9	1
Pullover	24	1	863	21	91
Dress	23	9	12	924	32
Coat	2	0	77	18	903

The diagonal entries show the correctly classified test samples. Trouser has the strongest class-wise result with 982 correct predictions out of 1,000. The largest confusion is between Pullover and Coat: 91 Pullover samples were predicted as Coat, while 77 Coat samples were predicted as Pullover. This indicates that these visually similar upper-body categories remain the most difficult pair for the selected FCNN.

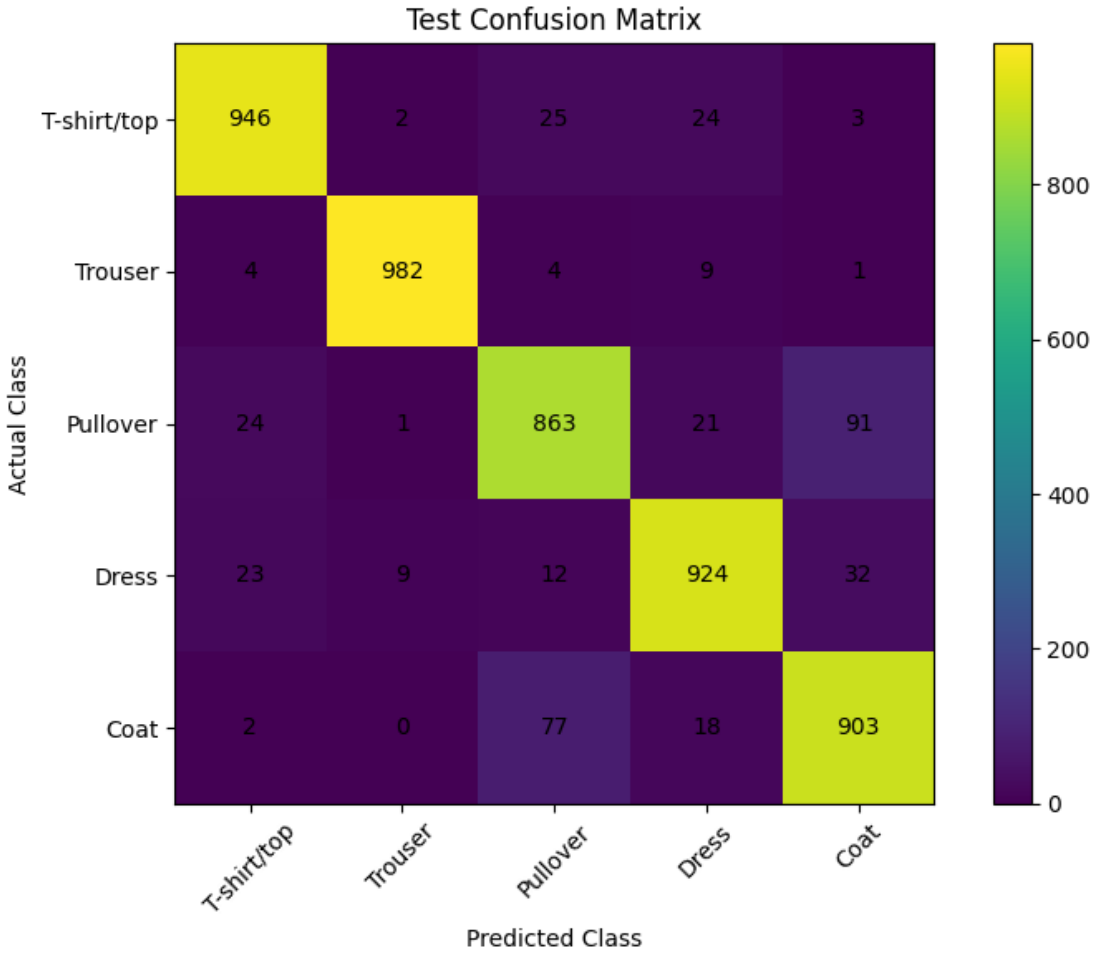


Figure 4: Test confusion matrix for the selected Architecture_2 + Adam model.

7.3 Training Confusion Matrix

Actual \ Predicted	T-shirt/top	Trouser	Pullover	Dress	Coat
T-shirt/top	4800	0	0	0	0
Trouser	0	4800	0	0	0
Pullover	0	0	4800	0	0
Dress	0	0	0	4800	0
Coat	0	0	0	0	4800

The training confusion matrix is perfectly diagonal, indicating that the selected model classified all 24,000 training samples correctly. This is consistent with the reported 100.00% training accuracy.

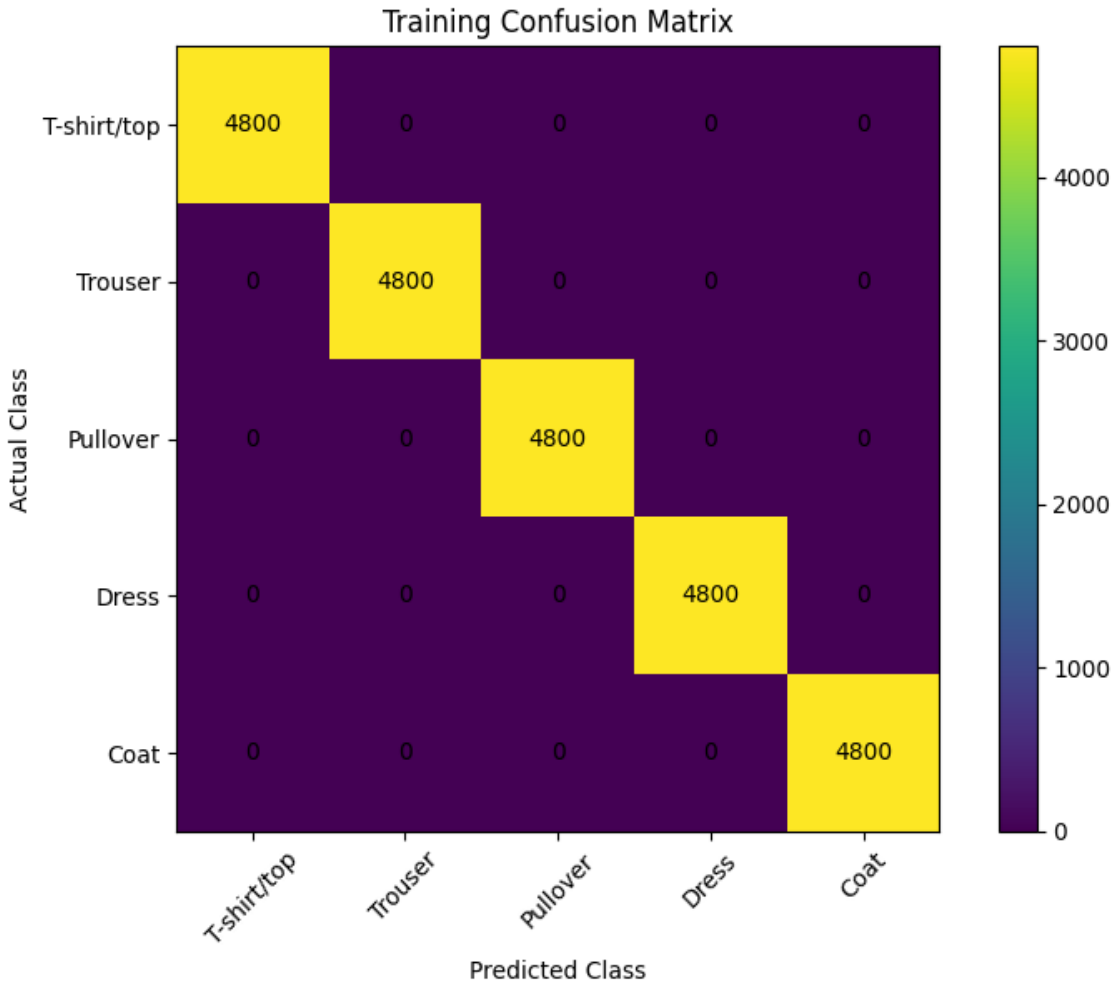


Figure 5: Training confusion matrix for the selected Architecture_2 + Adam model.

8. Discussion

The updated experiment shows that validation performance depends strongly on both network architecture and optimizer. Architecture_2 with Adam produced the highest validation accuracy (92.05%), while Architecture_2 with RMSProp was a close second at 91.53%. Architecture_1 with RMSProp achieved 91.52%, and Architecture_3 with RMSProp achieved 91.28%.

Batch GD reached the numerical convergence threshold in only two epochs for all three architectures, but its validation accuracy remained close to 20%, which is approximately chance level for a five-class classification problem. This demonstrates that the chosen convergence rule measures the change in training error, not whether the classifier has reached a useful predictive solution.

The deeper Architecture_3 did not provide the best validation accuracy. Although it achieved high training accuracy with several optimizers, its best validation accuracy was 91.28% with RMSProp, below the 92.05% achieved by Architecture_2 with Adam. Therefore, increasing depth did not automatically improve generalization in this experiment.

The selected Adam model has a 100.00% training accuracy compared with 92.05% validation accuracy and 92.36% test accuracy. The gap indicates that the model fits the training data extremely well, while the validation and test results show strong but lower performance on unseen data.

9. Conclusion

This experiment compared three FCNN architectures and six optimization algorithms on a five-class Fashion-MNIST classification problem. The data-processing pipeline included class filtering, explicit label remapping, normalization of pixel values to [0, 1], a stratified 80:20 train-validation split, conversion of arrays to PyTorch tensors, and creation of TensorDataset objects. The final data sizes were 24,000 training samples, 6,000 validation samples, and 5,000 test samples.

The best configuration was Architecture_2 with hidden layers [256, 128, 64, 32] and the Adam optimizer. It achieved 100.00% training accuracy, 92.05% validation accuracy, and 92.36% test accuracy, with convergence after 244 epochs. The results show that optimizer choice and architecture both affect convergence and generalization, and that numerical convergence alone should not be treated as evidence of good classification performance.